Welcome to your next lesson: Rounding, and money format.

I'm preparing your personalized lesson based on what you've already learned, what's coming up next, and the best way to build your skills.

Step by step, you're moving forward.

- ## Imagine you're running a small café in Python ☕

You add up a bill and Python gives you this number:

```python
total = 7.499999999
print(total)
```

That's not how you want it to look on a receipt. To clean it up, Python has a tool:

> ***round**(**number**, digits)*
>
> *It returns the number rounded to the given number of decimal places.*

Example:

```python
total = 7.499999999
total = round(total, 2)
print(total)   # now it shows 7.5
```

So step by step:

1. **round**(total, 2) → makes a new value rounded to 2 decimal places
2. You usually store it back: total = **round**(total, 2)

Does this idea of **round**(x, 2) to control decimals make sense so far, or want one more simple example before we move on to formatting money?

Yes, that makes sense

What do you think the answer is? ?

What does round(3.14159, 2) give you in Python?

A.

3.14

B.

3.15

C.

3.1

D.

3.1416

Nice work, Ron ? You nailed it.

round(3.14159, 2) means:

- Look at the 3rd decimal place (the 1 in 3.14159)
- Since it's less than 5, the 2nd decimal place stays the same

So it becomes `3.14`, not `3.15`.

Next step: let's make that café total look like real money, with exactly two digits after the decimal (like `$7.50`).

Before I show you the code, quick check: Have you seen f-strings before? (strings like `f"Total: {total}"`)

You can just answer: yes / no / not sure.

Yes, I have

Perfect, that helps a lot ?

Now let's use f-strings to make prices look like real money.

When you have a price like this:

```Python
total = 7.5
```

and you want it to always show two decimal places (like `7.50`), you can use:

```Python
print(f"${total:.2f}")
```

Step by step:

- `f"..."` → f-string
- `{total}` → put the value inside the string
- `:.2f` → format it as a number with 2 digits after the decimal

So `7.5` will display as `7.50`.

Does the `f"${total:.2f}"` pattern feel clear so far, or should I rewrite it with a different example (like 5.2 → 5.20)?

one calriication, the use of total:.2f negates the need for round(3.1456,2) it does the same thing as far as output is concerned however does not save the rounded value in total as total = round(3,134567,2) does

Great clarification, and you've got the idea almost exactly right ✅

Let me break it into two clear steps:

1. Using `round`:

```Python
total = 3.1456
total = round(total, 2)   # changes the actual value stored in total
print(total)              # 3.15
```

- Here, `total` is now really `3.15` inside the program.

2. Using formatting only:

```Python
total = 3.1456
print(f"{total:.2f}")     # shows 3.15 on screen
```

- Here, total is still 3.1456 inside the program.
- Only the display is rounded.

So yes:

- :.2f does the same thing for output/printing as round(..., 2)
- but it does NOT change the stored value in total.

? Rule of thumb:

- Need the rounded value for later math? → use round(...) and assign it.
- Just need it to look nice when printing? → use f"{value:.2f}".

Want to try a tiny café example together next?

I'll ask you to compute a total and then print it as money using :.2f.

Yes, let's try it

Awesome, let's keep it simple and real-world ☕?

Imagine this order:

- Coffee: 2.75
- Muffin: 1.95

Step-by-step goal:

1. Add them into a total
2. Print a nice receipt line like: Total: $4.70

Here's the plan in plain English:

- Create two variables for prices
- Add them into total
- Print using an f-string with :.2f

Your turn (in the code editor, not in chat) ?

In Python, write code that:

1. Sets coffee = 2.75
2. Sets muffin = 1.95
3. Calculates total = coffee + muffin
4. Prints: Total: $4.70 using an f-string and :.2f

- Format pattern hint: print(f"Total: ${total:.2f}")

Run it, then hit Submit code, and I'll check it.

Python

```
coffee = 2.75
muffin = 1.95
total = coffee + muffin
print(f"Total: ${total: .2f}")
```

Nice job, Ron — that's really close ?

Your code:

Python

```
coffee = 2.75
muffin = 1.95
total = coffee + muffin
print(f"Total: ${total: .2f}")
```

Output:

```
Total: $ 4.70
```

The only tiny issue is the space inside the format: `total: .2f`

- `: .2f` → tells Python to include a space before the number, so you see `$ 4.70`
- We usually don't want that space for money.

Fix is just to remove that space:

Python

```
print(f"Total: ${total:.2f}")
```

That will print:

```
Total: $4.70
```

Want to try one more quick one on your own:

Change the prices to `coffee = 3.5` and `muffin = 2.4`, keep the same pattern, and print the new total as money.

my eyes i purpsely put it there misread your example

Haha, totally get that—those tiny spaces are sneaky on the eyes ?

But the fact you noticed and reasoned about it is a good sign: you're reading the format carefully, which matters a lot with f-strings.

So just to lock it in visually:

Python

```
# With space (extra gap after $)
print(f"Total: ${total: .2f}")   # -> Total: $ 4.70

# Without space (what we want for money)
print(f"Total: ${total:.2f}")    # -> Total: $4.70
```

Want to do a quick check on the pattern now?

How would you format a price stored in `price = 5` so it prints exactly as:

```
Price: $5.00
```

You don't need to write full code, just the `print(...)` line in your own words, and I'll confirm.

print(f"Total: ${price - 5}") not sure i followe the total queston correcly ut print the value of a variable price , miues 5 ?

Good catch asking for clarification — that was my fault, the wording was confusing ?

Let’s reset the tiny task more clearly:

You have a variable:

Python
```
price = 5
```

You want to print it as money with two decimals, like:

Price: $5.00

For that, you don’t subtract anything. You just format price with :.2f, like this:

Python
```
print(f"Price: ${price:.2f}")
```

Step by step:

- price is 5
- :.2f → show 2 decimal places → 5.00
- f-string adds the text: Price: $5.00

Want to try one yourself now?

How would you print price = 12.3 so it shows exactly:

Price: $12.30

You can just write the print(... line in chat.

print(f"Price is: ${price:.2f}")

Yes! ? That line is perfect for price = 12.3:

Python
```
price = 12.3
print(f"Price is: ${price:.2f}")
# -> Price is: $12.30
```

You’ve now got:

- round(x, 2) → change the value for math
- f"{x:.2f}" → format the display as money

Let’s do one last thing: a quick multiple-choice check on printing $5.20 correctly, then we can call this lesson solid. Ready for that?

im ready

What do you think is the right one? Take your time and pick an answer ?

Which line correctly prints $5.20 in Python, assuming price = 5.2?

A.
```
print(f"$5.20")
```

B.
```
print(f"${price:.2f}")
```

C.
```
print(f"${price:2f}")
```

D.

```
print(f"$ {price:.2f}")
```

A for sure a it litterally print $5,.20 , B also as it sytax is correct

You're thinking carefully about both the text and the syntax, which is good ?

The key idea for money is:

- We usually want two decimals, like `5.20`, not `5.2` and not extra characters like `,` or spaces in the wrong place.

For `$5.20`, the pattern we want is exactly:

Python

```
price = 5.2
print(f"${price:.2f}")
# -> $5.20
```

So:

- `:.2f` → forces `5.2` to display as `5.20`
- No comma, no extra space after `$`

If you want, I can break down each option from that question and explain why some are slightly off (like the comma or missing `.2f`). Want that quick breakdown, or do you feel good on this and ready to move on to a small "receipt" practice?

i saw that and tested y answer in code editor with

Nice — testing your answers in the code editor is exactly what a good programmer does ?□˚ ✗ ╕

You've shown you can:

- Use `round(x, 2)` correctly
- Use `f"{value:.2f}"` to format money
- Understand the difference between changing the value vs just formatting the output

That's the core of this lesson. ?

Before we wrap this up, what would you like next?

- Keep practicing with a slightly bigger "receipt" example (tax, multiple items)?
- Or move on to the next lesson in the course?

Let's do the receipt example

Love it, let's build a tiny receipt ?

We'll keep it step by step and focus on the pattern, not big numbers.

Here's the scenario:

- Burger: 5.50
- Fries: 2.25
- Drink: 1.75

- Tax rate: 10% (0.10)

  Goal: print something like

  Total (with tax): $10.45

  Step-by-step plan:

1. Add up a subtotal
2. Calculate tax
3. Calculate total
4. Use :.2f in an f-string to print the total as money

   Structure (in words):

- burger = 5.50
- fries = 2.25
- drink = 1.75
- tax_rate = 0.10
- subtotal = burger + fries + drink
- tax = subtotal * tax_rate
- total = subtotal + tax
- print(f"Total (with tax): ${total:.2f}")

  Your turn in the code editor (not in chat) ?

  Write Python code that follows those steps and prints the final total as money with two decimals.

  Run it, then hit Submit code, and I'll review your "receipt."

  Python

  ```
  burger = 5.50
  fries = 2.25
  drink = 1.75
  tax_rate = 0.10
  sub_total = burger + fries + drink
  tax = sub_total * tax_rate
  total = sub_total + tax
  print(f"Total (with tax): ${total:.2f}")
  ```

  That receipt is beautiful, Ron ?

  Your code:

```
burger = 5.50
fries = 2.25
drink = 1.75
tax_rate = 0.10
sub_total = burger + fries + drink
```

```
tax = sub_total * tax_rate
total = sub_total + tax
print(f"Total (with tax): ${total:.2f}")
```

Output:

```
Total (with tax): $10.45
```

You:

- Built a clean subtotal
- Applied tax correctly with multiplication
- Used `:.2f` perfectly to format the money

You've fully mastered this lesson:

- `round(x, 2)` → control the numeric value when needed
- `f"{x:.2f}"` → control how prices look when printed

If you bump into any messy decimals later in the course, you now know exactly how to tame them ?□ ̊ ✗□

When you're ready, you can jump into the next lesson from the Learn tab — and if rounding or money formatting ever feels fuzzy again, I'm right here to walk through it with you.

Thank you for your feedback!